

Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: 09.03.02 Информационные системы и
технологии

ОТЧЕТ

по проектной практике

Студенты: Сологуб Б.Д. Перевозчикков Р.Д. Группа: 251-337

Место прохождения практики: Московский Политех, кафедра Информатика
и информационные технологии

Отчет принят с оценкой _____ Дата _____

Руководитель практики: Дагаев Александр Евгеньевич

Москва 2026

Оглавление

ВВЕДЕНИЕ.....	3
1. БАЗОВАЯ ЧАСТЬ ПРАКТИКИ	4
1.1 Настройка Git и репозитория	4
1.2 Документирование в Markdown	5
1.3 Создание статического веб-сайта	6
2. ВАРИАТИВНАЯ ЧАСТЬ ПРАКТИКИ	7
2.1 Постановка задачи	7
2.2 Архитектура приложения	8
2.3 Тестирование	11
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	14
ПРИЛОЖЕНИЕ А	15
ПРИЛОЖЕНИЕ Б	15

ВВЕДЕНИЕ

Проектная практика является важным этапом учебного процесса, направленным на закрепление теоретических знаний по дисциплине «Технология прикладного программирования» и приобретение практических навыков разработки реального программного продукта с применением современных инструментов и технологий.

В рамках практики поставлены две основные цели: освоить базовые инструменты профессионального разработчика — систему контроля версий Git, язык разметки Markdown и основы веб-вёрстки, а также реализовать самостоятельный программный проект — настольное WPF-приложение для управления семейным бюджетом.

Выбор темы вариативной части обусловлен практической значимостью задачи управления личными финансами. Большинство семей ведут учёт расходов вручную или с помощью электронных таблиц, что неудобно и не позволяет получить наглядную аналитику. Разработанное приложение решает эту проблему, предоставляя удобный инструмент учёта доходов и расходов каждого члена семьи, постановки и отслеживания целей накоплений, а также визуализации финансовой статистики.

Практика состояла из базовой части, включавшей настройку Git-репозитория, оформление документации в Markdown и создание статического веб-сайта проекта, и вариативной части — проектирования и реализации полнофункционального WPF-приложения на платформе .NET 8. Общая трудоёмкость составила 72 часа, работа выполнена индивидуально.

Результатом практики стало работающее настольное приложение «Семейный Бюджет», техническая документация к нему и веб-страница с описанием проекта. Ссылка на проект:

<https://github.com/m0n1ks/practice-2026-1/blob/master/kursac.slnx>

1. БАЗОВАЯ ЧАСТЬ ПРАКТИКИ

1.1 Настройка Git и репозитория

Создан личный репозиторий на платформе GitHub для размещения всех материалов практики. Ссылка на репозиторий:

```
https://github.com/SologubBogdan/family-budget-wpf
```

В процессе работы освоены основные команды Git:

- `git clone` — клонирование удалённого репозитория на локальный компьютер;
- `git add` — добавление изменённых файлов в индекс (staging area);
- `git commit` — фиксация изменений с описательным комментарием;
- `git push` — отправка локальных коммитов на удалённый сервер;
- `git checkout -b` — создание новой ветки и переключение на неё;
- `git merge` — слияние ветки разработки с основной веткой `main`.

Разработка велась с регулярной фиксацией изменений после завершения каждого функционального блока. Структура репозитория соответствует заданию и организована по следующим папкам:

- `docs/` — техническая документация в формате Markdown;
- `reports/` — итоговый отчёт по практике;
- `site/` — статический веб-сайт с описанием проекта;
- `src/` — исходный код WPF-приложения;
- `task/` — постановка задачи и требования к проекту.

1.2 Документирование в Markdown

Изучен синтаксис языка разметки Markdown. Вся техническая документация проекта оформлена с использованием возможностей Markdown:

- заголовки разных уровней (`# H1`, `## H2`, `### H3`);
- нумерованные и маркированные списки;

- таблицы для представления структурированных данных;
- блоки кода с указанием языка программирования (````csharp`);
- ссылки на внешние ресурсы и изображения;
- жирный и курсивный текст для выделения ключевых понятий.

Итоговая документация размещена в файле `docs/README.md` и содержит следующие разделы: описание функциональных требований к приложению, архитектуру проекта и описание модели данных, руководство по сборке и запуску из исходного кода, описание пользовательского интерфейса каждой страницы, хронологию этапов разработки.

1.3 Создание статического веб-сайта

Разработан статический сайт с использованием HTML5 и CSS3 для презентации проекта. Сайт содержит 5 страниц:

- Главная — аннотация проекта, цветовая палитра интерфейса и скриншот главного экрана приложения;
- О проекте — цели разработки, описание архитектуры, диаграмма классов на чистом CSS, схема навигации между страницами приложения;
- Разработчик — информация об авторе, применённые технологии и личный вклад в проект;
- Журнал — три записи о ходе разработки с описанием выполненных этапов;
- Ресурсы — ссылки на использованную документацию и учебные материалы.

Сайт размещён в папке `site/` репозитория. Цветовая схема сайта соответствует теме приложения: тёмный фон с розово-фиолетовыми акцентами. Схема архитектуры выполнена средствами чистого CSS с

использованием flexbox-разметки без применения изображений. Ссылка на сайт:

<https://github.com/SologubBogdan/family-budget-wpf/tree/main/site>

2. ВАРИАТИВНАЯ ЧАСТЬ ПРАКТИКИ

2.1 Постановка задачи

Задача: разработать настольное WPF-приложение «Семейный Бюджет» для комплексного управления финансами домохозяйства. Приложение должно поддерживать несколько профилей членов семьи, вести учёт доходов и расходов по каждому участнику, хранить цели накоплений с задачами и предоставлять аналитику по финансовым данным.

Для реализации выбраны следующие технологии:

- Язык программирования: C# 12 на платформе .NET 8;
- UI-фреймворк: Windows Presentation Foundation (WPF);
- Разметка интерфейса: XAML с использованием стилей, шаблонов и анимаций;
- Хранение данных: оперативная память с предзаполнением (in-memory коллекции `ObservableCollection<T>`);
- Среда разработки: Microsoft Visual Studio 2022 Community.

Функциональные требования к приложению:

1. управление профилями членов семьи — добавление, просмотр и удаление;
2. учёт финансовых операций (доходы и расходы) с привязкой к члену семьи, категорией, описанием и датой;
3. постановка целей накоплений с указанием целевой суммы, срока и иконки;
4. ведение задач (чеклистов) для каждой цели накоплений;
5. аналитический экран с визуализацией расходов по категориям и по членам семьи;
6. навигация между разделами через нижнюю панель в стиле мобильного приложения.

2.2 Архитектура приложения

Проект организован по слоям. Папка Models содержит все классы предметной области и статический репозиторий AppData. Папка Converters содержит конвертеры значений для привязки данных XAML. Папка Pages содержит пять пар файлов (.xaml + .xaml.cs) для страниц приложения. Корневой уровень содержит App.xaml с глобальными стилями и MainWindow.xaml с навигационной оболочкой.

Основные классы модели данных:

Класс FamilyMember описывает члена семьи. Реализует интерфейс INotifyPropertyChanged для поддержки привязки данных. Свойства: Id (int), Name (string), Role (string), AvatarColor (строка HEX-цвета), Emoji (string). Вычисляемое свойство Initials возвращает первую букву имени.

Класс Transaction описывает финансовую операцию. Перечисление TransactionType определяет тип — Income (доход) или Expense (расход). Свойства: Id, MemberId, MemberName, MemberColor, Amount (decimal), Type, Category, CategoryIcon, Description, Date. Вычисляемые свойства FormattedAmount и AmountColor упрощают привязку к элементам интерфейса.

Класс SavingsGoal описывает цель накоплений. Свойство CurrentAmount является полным и при изменении уведомляет зависимые вычисляемые свойства: Progress, ProgressText, Remaining, IsComplete. Это обеспечивает автоматическое обновление полос прогресса при пополнении цели.

Класс `GoalTask` описывает задачу, связанную с конкретной целью накоплений. Свойства: `Id`, `GoalId`, `Title`, `IsDone`.

Статический класс `AppData` является центральным репозиторием. Хранит `ObservableCollection<T>` для каждой сущности и предоставляет методы-фабрики: `AddMember()`, `AddTransaction()`, `AddGoal()`, `AddGoalTask()`, а также методы удаления. Инициализируется статическим конструктором, вызывающим `SeedData()` с предзаполнением демонстрационными данными. Пример реализации метода `AddTransaction`:

```
public static void AddTransaction(int memberId, string memberName,
    string memberColor, decimal amount, TransactionType type,
    string category, string icon, string description, DateTime
date)
{
    Transactions.Add(new Transaction
    {
        Id = _tId++, MemberId = memberId,
        MemberName = memberName, MemberColor = memberColor,
        Amount = amount, Type = type,
        Category = category, CategoryIcon = icon,
        Description = description, Date = date
    });
}
```

Главное окно `MainWindow` реализует нестандартный оконный хром (`WindowStyle="None"`, `AllowsTransparency="True"`). Окно имеет фиксированный размер 430×860 пикселей, имитирующий пропорции экрана смартфона. Перетаскивание окна реализовано через обработчик `MouseDown` с вызовом `DragMove()`. Переключение страниц выполняется методом `ShowPage()` с ленивой инициализацией экземпляров страниц:

```
private void ShowPage(string page)
{
    UserControl? uc = page switch
    {
        "Dashboard" => _dashPage ??= new DashboardPage(),
```

```

        "Family"      => _famPage    ??= new FamilyPage(),
        "Transactions" => _transPage ??= new TransactionsPage(),
        "Goals"       => _goalsPage  ??= new GoalsPage(),
        "Analytics"   => _analPage   ??= new AnalyticsPage(),
        —            => null
    };
    if (uc == null) return;
    if (uc is DashboardPage dp) dp.Refresh();
    if (uc is FamilyPage fp) fp.Refresh();
    if (uc is TransactionsPage tp) tp.Refresh();
    if (uc is GoalsPage gp) gp.Refresh();
    if (uc is AnalyticsPage ap) ap.Refresh();
    PageContent.Content = uc;
    SetActiveNav(page);
}

```

Глобальные стили определены в App.xaml. Для всех интерактивных кнопок реализованы анимации через `ControlTemplate.Triggers` с использованием `DoubleAnimation` и `ScaleTransform`. При наведении кнопка масштабируется до 104%, при нажатии — уменьшается до 95%. FAB-кнопки (Floating Action Button) дополнительно имеют розовый эффект свечения `DropShadowEffect` с `Color="#E91E8C"`.

2.3 Тестирование

Тестирование проводилось вручную по сценариям использования. Приложение запускается с предзаполненными данными: четыре члена семьи (Папа, Мама, Сын, Дочь), 16 финансовых операций различных категорий, 5 целей накоплений.

Главный экран (Dashboard) при запуске отображает:

- общий баланс семьи: 153 300 Р (суммарные доходы минус суммарные расходы);
- суммарные доходы всех членов: 175 500 Р;
- суммарные расходы: 57 800 Р;
- количество членов семьи: 4; количество целей: 5;

- последние 8 операций в хронологическом порядке.

Проверка страницы «Семья»:

- отображаются карточки всех четырёх членов семьи с индивидуальными аватарами;
- для каждого члена корректно вычисляются и отображаются доходы, расходы и баланс;
- при нажатии FAV открывается форма добавления нового члена семьи;
- после заполнения полей «Имя» и «Роль» и выбора цвета аватара новый член добавляется в список;
- кнопка удаления (☐) вызывает диалог подтверждения; после подтверждения член семьи удаляется вместе со всеми его транзакциями.

Проверка страницы «Операции»:

- в режиме «Все» отображаются 16 транзакций, отсортированных по дате (новые сверху);
- фильтр «Доходы» корректно оставляет только 7 записей с положительными суммами (зелёный цвет);
- фильтр «Расходы» отображает 9 записей с красными суммами;
- выпадающий список фильтрации по члену семьи корректно ограничивает выборку;
- добавление новой операции через FAV-форму отображает запись в списке немедленно;
- кнопка «X» удаляет операцию из списка без перезапуска приложения.

Проверка страницы «Цели»:

- все пять целей отображаются с корректными полосами прогресса;

- пополнение цели «Отпуск в Турции» на 10 000 Р: прогресс меняется с 31% до 38%;
- после достижения целевой суммы появляется поздравительное сообщение;
- добавление задачи в чеклист и установка флажка IsDone работают корректно;
- удаление цели удаляет и все связанные задачи.

Проверка страницы «Аналитика»:

- горизонтальные полосы доходов и расходов корректно масштабируются относительно максимального значения;
- топ-8 категорий расходов отображаются в порядке убывания суммы с иконками и полосами пропорциональной ширины;
- для каждого члена семьи корректно отображаются доходы, расходы и цветовой индикатор баланса;
- полосы прогресса целей наследуют индивидуальные цвета соответствующих целей.

Все проверенные сценарии отработали корректно. Критических ошибок и утечек данных между страницами не обнаружено. Анимации кнопок работают плавно, без артефактов.

ЗАКЛЮЧЕНИЕ

В ходе проектной практики успешно выполнены все поставленные задачи.

Освоены инструменты разработки:

- система контроля версий Git (clone, add, commit, push, checkout -b, merge);
- язык разметки Markdown для оформления технической документации;
- HTML5 и CSS3 для создания статического презентационного сайта.

Разработано WPF-приложение «Семейный Бюджет»:

- спроектирована архитектура на основе паттерна MVP с использованием ObservableCollection<T> и INotifyPropertyChanged;
- реализованы пять функциональных страниц: главный экран, семья, операции, цели накоплений и аналитика;
- создан уникальный пользовательский интерфейс в стиле мобильного приложения с тёмной розово-чёрной цветовой схемой;
- реализованы анимации кнопок на основе WPF Storyboard (масштабирование, свечение);
- разработана система конвертеров значений для привязки данных в XAML.

Подготовлена документация:

- техническое описание проекта в формате Markdown;
- статический веб-сайт с диаграммами архитектуры;
- итоговый отчёт по проектной практике.

Проект демонстрирует практическое применение технологий .NET 8 и WPF для создания современных настольных приложений. Приобретены

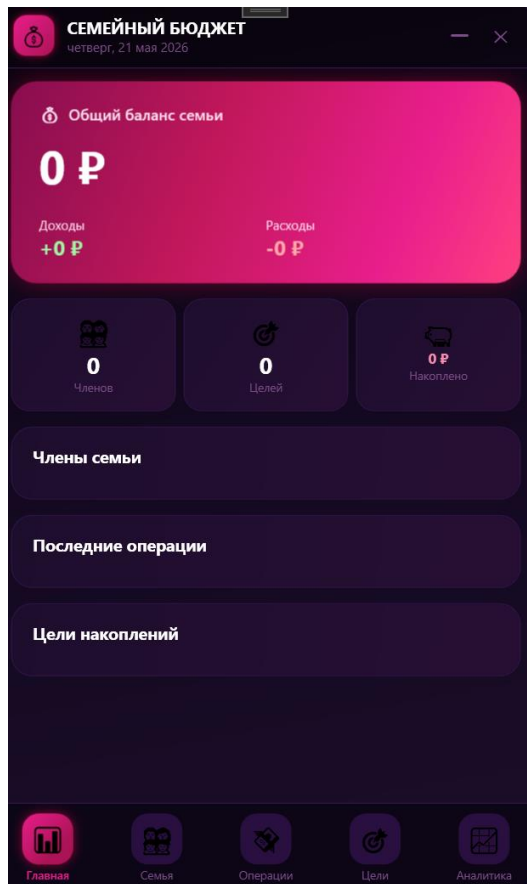
навыки проектирования модели данных, реализации привязки данных в XAML, создания анимированных стилей и шаблонов элементов управления. Полученный опыт служит основой для дальнейшего изучения более сложных архитектурных паттернов (MVVM, DI) и систем управления базами данных.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Документация Microsoft по Windows Presentation Foundation:
<https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/overview/>
2. Документация Microsoft по привязке данных в WPF:
<https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/data/data-binding-overview>
3. Документация Microsoft по анимациям WPF:
<https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/graphics-multimedia/animation-overview>
4. Документация Microsoft по C# 12: <https://learn.microsoft.com/ru-ru/dotnet/csharp/whats-new/csharp-12>
5. Макдональд М. WPF: Windows Presentation Foundation в .NET 4.5 с примерами на C# 5.0. — М.: Вильямс, 2013. — 1024 с.
6. Нейгел К., Ивсен Б. C# 8.0 и .NET Core 3.0. Современные кросс-платформенные разработки. — М.: Диалектика, 2020. — 1240 с.
7. Рихтер Д. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5. — 4-е изд. — СПб.: Питер, 2013. — 896 с.
8. Руководство по Git: <https://git-scm.com/book/ru/v2>
9. Руководство по Markdown: <https://doka.guide/tools/markdown/>
10. GitHub — платформа для размещения кода: <https://github.com>

ПРИЛОЖЕНИЕ А

Скрины экранов приложения:





СЕМЕЙНЫЙ БЮДЖЕТ
четверг, 21 мая 2026



Операции



Новая операция



↑ Доход

↓ Расход

Сумма (₽)

Описание

Член семьи

Категория

(Зарплата, , Зарплата)

☒ Сохранить операцию

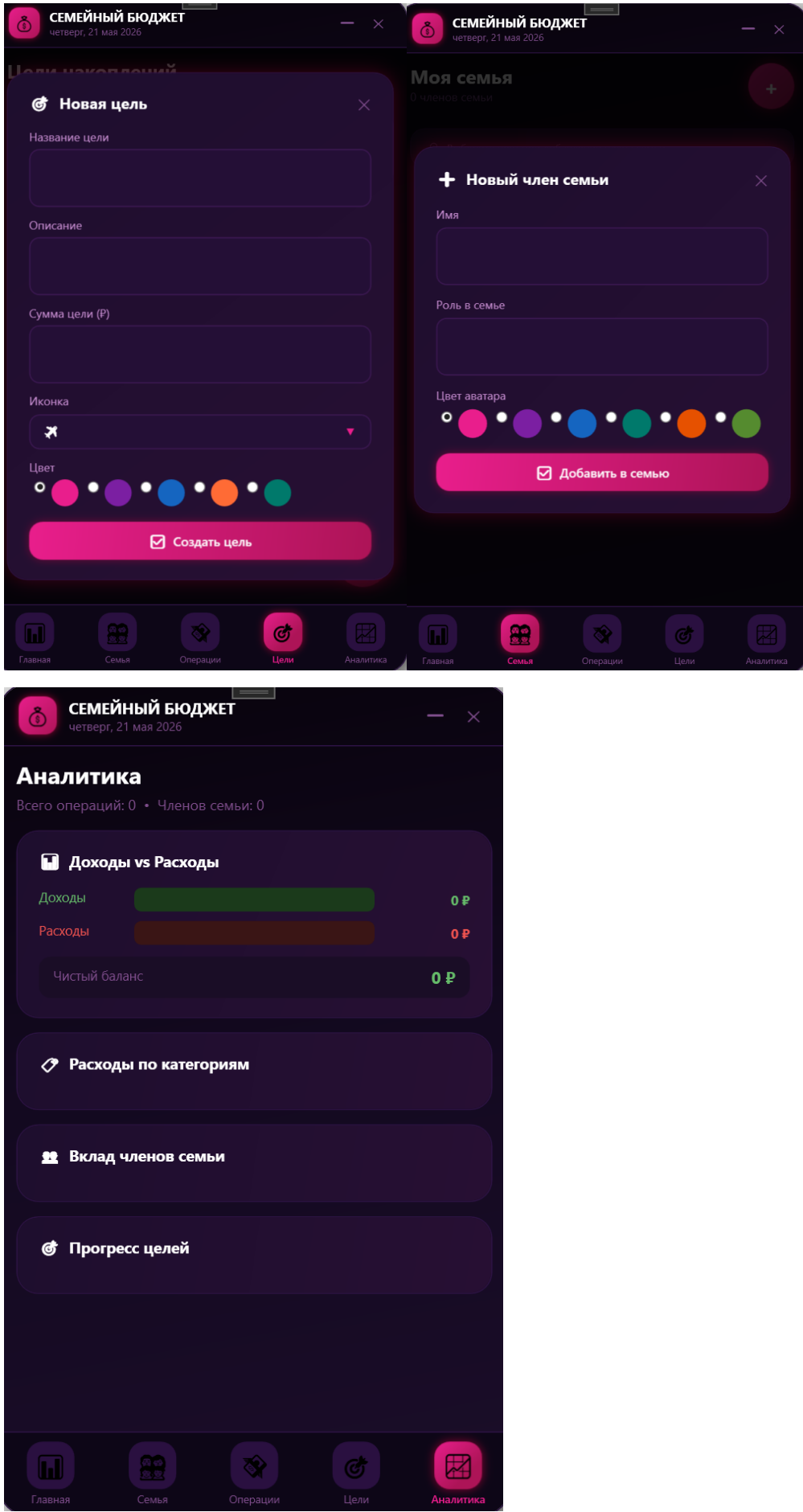

Главная


Семья


Операции


Цели


Аналитика



ПРИЛОЖЕНИЕ Б

Листинг ключевых фрагментов кода

Файл: Models/AppData.cs — класс SavingsGoal:

```
public class SavingsGoal : INotifyPropertyChanged
{
    private decimal _currentAmount;
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal TargetAmount { get; set; }
    public decimal CurrentAmount
    {
        get => _currentAmount;
        set {
            _currentAmount = value;
            PC(nameof(CurrentAmount));
            PC(nameof(Progress));
            PC(nameof(IsComplete));
        }
    }
    public double Progress =>
        TargetAmount > 0
        ? Math.Min(1.0, (double)(CurrentAmount / TargetAmount))
        : 0;
    public bool IsComplete => CurrentAmount >= TargetAmount;
    public event PropertyChangedEventHandler? PropertyChanged;
    void PC(string n) => PropertyChanged?.Invoke(this,
        new PropertyChangedEventArgs(n));
}
```

Файл: Converters/ValueConverters.cs — HexToBrushConverter:

```
public class HexToBrushConverter : IValueConverter
{
    public object Convert(object v, Type t,
        object p, CultureInfo c)
    {
        try {
            return new SolidColorBrush(
                (Color)ColorConverter
                    .ConvertFromString(v?.ToString()
                        ?? "#E91E8C"));
        }
        catch { return new SolidColorBrush(Colors.HotPink); }
    }
    public object ConvertBack(object v, Type t,
        object p, CultureInfo c)
```

```

=> throw new NotImplementedException();
}

```

Файл: Pages/TransactionsPage.xaml.cs — фильтрация операций:

```

private void ApplyFilter()
{
    var q = AppData.Transactions.AsEnumerable();
    if (_filter == "Income")
        q = q.Where(t => t.Type == TransactionType.Income);
    if (_filter == "Expense")
        q = q.Where(t => t.Type == TransactionType.Expense);
    if (_memberFilter > 0)
        q = q.Where(t => t.MemberId == _memberFilter);
    var list = q.OrderByDescending(t => t.Date).ToList();
    TransList.ItemsSource = null;
    TransList.ItemsSource = list;
    TxtEmpty.Visibility = list.Count == 0
        ? Visibility.Visible : Visibility.Collapsed;
}

```

Файл: MainWindow.xaml — визуальная составляющая:

```

<Window x:Class="FamilyBudget.MainWindow"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Семейный Бюджет"
    Width="430" Height="700"
    MinWidth="430" MinHeight="700"
    MaxWidth="430" MaxHeight="700"
    WindowStyle="None"
    AllowsTransparency="True"
    Background="Transparent"
    ResizeMode="CanMinimize"
    WindowStartupLocation="CenterScreen">

    <Window.Resources>
        <Style x:Key="NavBtn" TargetType="Button">
            <Setter Property="Background" Value="Transparent"/>
            <Setter Property="BorderThickness" Value="0"/>
            <Setter Property="Cursor" Value="Hand"/>
            <Setter Property="Width" Value="80"/>
            <Setter Property="RenderTransformOrigin"
Value="0.5,0.5"/>
            <Setter Property="RenderTransform">
                <Setter.Value>

```

```

        <ScaleTransform/>
    </Setter.Value>
</Setter>
<Setter Property="Template">
    <Setter.Value>
        <ControlTemplate TargetType="Button">
            <Border Background="Transparent"
Padding="4,6">
                <ContentPresenter
HorizontalAlignment="Center" VerticalAlignment="Center"/>
            </Border>
            <ControlTemplate.Triggers>
                <EventTrigger
RoutedEvent="PreviewMouseDown">
                    <BeginStoryboard>
                        <Storyboard>
                            <DoubleAnimation
Storyboard.TargetProperty="(RenderTransform).(ScaleTransform.Scale
X)" To="0.88" Duration="0:0:0.08"/>
                            <DoubleAnimation
Storyboard.TargetProperty="(RenderTransform).(ScaleTransform.Scale
Y)" To="0.88" Duration="0:0:0.08"/>
                        </Storyboard>
                    </BeginStoryboard>
                </EventTrigger>
                <EventTrigger
RoutedEvent="PreviewMouseUp">
                    <BeginStoryboard>
                        <Storyboard>
                            <DoubleAnimation
Storyboard.TargetProperty="(RenderTransform).(ScaleTransform.Scale
X)" To="1.0" Duration="0:0:0.08"/>
                            <DoubleAnimation
Storyboard.TargetProperty="(RenderTransform).(ScaleTransform.Scale
Y)" To="1.0" Duration="0:0:0.08"/>
                        </Storyboard>
                    </BeginStoryboard>
                </EventTrigger>
            </ControlTemplate.Triggers>
        </ControlTemplate>
    </Setter.Value>
</Setter>
</Style>
</Window.Resources>
<Border Background="{StaticResource BgGradBrush}"
    CornerRadius="24"
    BorderBrush="#2E1445"
    BorderThickness="1">
    <Border.Effect>

```

```

        <DropShadowEffect Color="#000000" BlurRadius="40"
ShadowDepth="10" Opacity="0.8"/>
    </Border.Effect>

    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="52"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="70"/>
        </Grid.RowDefinitions>
        <Border Grid.Row="0" Background="Transparent"
            MouseDown="TitleBar_MouseDown"
            CornerRadius="24,24,0,0">
            <Grid Margin="16,0">
                <StackPanel Orientation="Horizontal"
VerticalAlignment="Center">
                    <Border Width="32" Height="32"
CornerRadius="10"
                        Background="{StaticResource
PrimaryGradBrush}"
                        Margin="0,0,10,0">
                            <Border.Effect>
                                <DropShadowEffect Color="#E91E8C"
BlurRadius="10" ShadowDepth="0" Opacity="0.7"/>
                            </Border.Effect>
                            <TextBlock Text="§" FontSize="16"
HorizontalAlignment="Center" VerticalAlignment="Center"/>
                        </Border>
                        <StackPanel VerticalAlignment="Center">
                            <TextBlock Text="СЕМЕЙНЫЙ БЮДЖЕТ"
                                Foreground="White"
FontSize="13" FontWeight="Bold"
                                    FontFamily="Segoe UI"/>
                            <TextBlock x:Name="TxtDate"
                                Foreground="{StaticResource TextMutedBrush}"
                                    FontSize="10"
                                FontFamily="Segoe UI"/>
                        </StackPanel>
                    </StackPanel>

                    <!-- Window controls -->
                    <StackPanel Orientation="Horizontal"
HorizontalAlignment="Right" VerticalAlignment="Center">
                        <Button Style="{StaticResource FlatBtn}"
Width="28" Height="28" Margin="0,0,4,0"
                            Click="MinimizeBtn_Click"
ToolTip="Свернуть">

```



```

                                <Border Width="14" Height="2"
Background="{StaticResource TextMutedBrush}" CornerRadius="1"
VerticalAlignment="Center"/>
                                </Button>
                                <Button Style="{StaticResource FlatBtn}"
Width="28" Height="28"
                                Click="CloseBtn_Click"
ToolTip="Закреть">
                                <TextBlock Text="X"
Foreground="{StaticResource TextMutedBrush}" FontSize="13"
FontWeight="Bold"/>
                                </Button>
                                </StackPanel>
                                </Grid>
                                </Border>
                                <Border Grid.Row="0" Height="1"
VerticalAlignment="Bottom"
                                Background="{StaticResource
CardBorderBrush}"/>

                                <ContentPresenter x:Name="PageContent" Grid.Row="1"/>
                                <Border Grid.Row="2" Background="{StaticResource
NavBgBrush}"
                                BorderBrush="{StaticResource CardBorderBrush}"
                                BorderThickness="0,1,0,0"
                                CornerRadius="0,0,24,24">
                                <Grid>
                                <Grid.ColumnDefinitions>
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                <ColumnDefinition Width="*" />
                                </Grid.ColumnDefinitions>

                                <!-- Dashboard -->
                                <Button x:Name="NavDashboard" Grid.Column="0"
Style="{StaticResource NavBtn}"
                                Click="NavDashboard_Click">
                                <StackPanel HorizontalAlignment="Center">
                                <Border x:Name="BrdDash" Width="40"
Height="40" CornerRadius="13"
                                Background="#2A1040"
HorizontalAlignment="Center" Margin="0,0,0,2">
                                <TextBlock Text="📊" FontSize="20"
HorizontalAlignment="Center" VerticalAlignment="Center"/>
                                </Border>

```

```

                                <TextBlock x:Name="LblDash"
Text="Главная" FontSize="9" FontFamily="Segoe UI"
                                Foreground="{StaticResource
TextMutedBrush}"

HorizontalAlignment="Center"/>
                                </StackPanel>
                                </Button>

                                <!-- Family -->
                                <Button x:Name="NavFamily" Grid.Column="1"
Style="{StaticResource NavBtn}"
                                Click="NavFamily_Click">
                                <StackPanel HorizontalAlignment="Center">
                                <Border x:Name="BrdFam" Width="40"
Height="40" CornerRadius="13"
                                Background="#2A1040"
HorizontalAlignment="Center" Margin="0,0,0,2">
                                <TextBlock Text="👨👩👦👧"
FontSize="20"

HorizontalAlignment="Center" VerticalAlignment="Center"/>
                                </Border>
                                <TextBlock x:Name="LblFam"
Text="Семья" FontSize="9" FontFamily="Segoe UI"
                                Foreground="{StaticResource
TextMutedBrush}"

HorizontalAlignment="Center"/>
                                </StackPanel>
                                </Button>

                                <!-- Transactions -->
                                <Button x:Name="NavTrans" Grid.Column="2"
Style="{StaticResource NavBtn}"
                                Click="NavTrans_Click">
                                <StackPanel HorizontalAlignment="Center">
                                <Border x:Name="BrdTrans" Width="40"
Height="40" CornerRadius="13"
                                Background="#2A1040"
HorizontalAlignment="Center" Margin="0,0,0,2">
                                <TextBlock Text="💰" FontSize="20"

HorizontalAlignment="Center" VerticalAlignment="Center"/>
                                </Border>
                                <TextBlock x:Name="LblTrans"
Text="Операции" FontSize="9" FontFamily="Segoe UI"

```

```

                                Foreground="{StaticResource
TextMutedBrush}"

HorizontalAlignment="Center"/>
        </StackPanel>
    </Button>

    <!-- Goals -->
    <Button x:Name="NavGoals" Grid.Column="3"
Style="{StaticResource NavBtn}"
        Click="NavGoals_Click">
        <StackPanel HorizontalAlignment="Center">
            <Border x:Name="BrdGoals" Width="40"
Height="40" CornerRadius="13"
                                Background="#2A1040"
HorizontalAlignment="Center" Margin="0,0,0,2">
                <TextBlock Text="🎯" FontSize="20"

HorizontalAlignment="Center" VerticalAlignment="Center"/>
            </Border>
            <TextBlock x:Name="LblGoals"
Text="Цели" FontSize="9" FontFamily="Segoe UI"
                                Foreground="{StaticResource
TextMutedBrush}"

HorizontalAlignment="Center"/>
        </StackPanel>
    </Button>

    <!-- Analytics -->
    <Button x:Name="NavAnalytics" Grid.Column="4"
Style="{StaticResource NavBtn}"
        Click="NavAnalytics_Click">
        <StackPanel HorizontalAlignment="Center">
            <Border x:Name="BrdAnal" Width="40"
Height="40" CornerRadius="13"
                                Background="#2A1040"
HorizontalAlignment="Center" Margin="0,0,0,2">
                <TextBlock Text="📊" FontSize="20"

HorizontalAlignment="Center" VerticalAlignment="Center"/>
            </Border>
            <TextBlock x:Name="LblAnal"
Text="Аналитика" FontSize="9" FontFamily="Segoe UI"
                                Foreground="{StaticResource
TextMutedBrush}"

HorizontalAlignment="Center"/>

```

```

        </StackPanel>
        </Button>
    </Grid>
</Border>

</Grid>
</Border>
</Window>

```

Файл: MainWindow.xaml.cs — backend c# составляющая:

```

using System;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Media.Effects;
using FamilyBudget.Pages;

namespace FamilyBudget
{
    public partial class MainWindow : Window
    {
        private DashboardPage? _dashPage;
        private FamilyPage? _famPage;
        private TransactionsPage? _transPage;
        private GoalsPage? _goalsPage;
        private AnalyticsPage? _analPage;

        private static readonly LinearGradientBrush _activeBg =
new LinearGradientBrush(
    Color.FromRgb(0xE9, 0x1E, 0x8C),
    Color.FromRgb(0xAD, 0x14, 0x57), 45);
        private static readonly SolidColorBrush _mutedBg =
new SolidColorBrush(Color.FromRgb(0x2A, 0x10, 0x40));
        private static readonly SolidColorBrush _activeFg =
new SolidColorBrush(Color.FromRgb(0xE9, 0x1E, 0x8C));
        private static readonly SolidColorBrush _mutedFg =
new SolidColorBrush(Color.FromRgb(0x7E, 0x4D, 0x8B));
        private static readonly DropShadowEffect _pinkGlow = new
DropShadowEffect
        {
            Color = Color.FromRgb(0xE9, 0x1E, 0x8C),
            BlurRadius = 12, ShadowDepth = 0, Opacity = 0.65
        };

        public MainWindow()
    }
}

```

```

        {
            InitializeComponent();
            TxtDate.Text = DateTime.Now.ToString("dddd, dd MMMM
YYYY");
            ShowPage("Dashboard");
        }
        private void NavDashboard_Click(object s, RoutedEventArgs
e) => ShowPage("Dashboard");
        private void NavFamily_Click(object s, RoutedEventArgs e)
=> ShowPage("Family");
        private void NavTrans_Click(object s, RoutedEventArgs e)
=> ShowPage("Transactions");
        private void NavGoals_Click(object s, RoutedEventArgs e)
=> ShowPage("Goals");
        private void NavAnalytics_Click(object s, RoutedEventArgs
e) => ShowPage("Analytics");

        private void ShowPage(string page)
        {
            UserControl? uc = page switch
            {
                "Dashboard"    => _dashPage ??= new
DashboardPage(),
                "Family"       => _famPage ??= new FamilyPage(),
                "Transactions" => _transPage ??= new
TransactionsPage(),
                "Goals"        => _goalsPage ??= new GoalsPage(),
                "Analytics"    => _analPage ??= new
AnalyticsPage(),
                _               => null
            };

            if (uc == null) return;

            if (uc is DashboardPage dp) dp.Refresh();
            if (uc is FamilyPage fp) fp.Refresh();
            if (uc is TransactionsPage tp) tp.Refresh();
            if (uc is GoalsPage gp) gp.Refresh();
            if (uc is AnalyticsPage ap) ap.Refresh();

            PageContent.Content = uc;
            SetActiveNav(page);
        }

        private void SetActiveNav(string page)
        {
            var items = new[]
            {
                (Brd: BrdDash, Lbl: LblDash, Key: "Dashboard"),
            }
        }
    }

```

```

        (Brd: BrdFam, Lbl: LblFam, Key: "Family"),
        (Brd: BrdTrans, Lbl: LblTrans, Key:
"Transactions"),
        (Brd: BrdGoals, Lbl: LblGoals, Key: "Goals"),
        (Brd: BrdAnal, Lbl: LblAnal, Key: "Analytics"),
    };

    foreach (var item in items)
    {
        bool isActive = item.Key == page;
        item.Brd.Background = isActive ? _activeBg :
_mutedBg;
        item.Brd.Effect = isActive ? _pinkGlow :
null;
        item.Lbl.Foreground = isActive ? _activeFg :
_mutedFg;
        item.Lbl.FontWeight = isActive ?
FontWeights.SemiBold : FontWeights.Normal;
    }
}

private void TitleBar_MouseDown(object s,
System.Windows.Input.MouseButtonEventArgs e)
{
    if (e.LeftButton ==
System.Windows.Input.MouseButtonState.Pressed)
        DragMove();
}

private void MinimizeBtn_Click(object s, RoutedEventArgs
e)
    => WindowState = WindowState.Minimized;
private void CloseBtn_Click(object s, RoutedEventArgs e)
    => Close();
}
}

```